

PPO vs TROLL Comparison on Atari Games

Based on article: [Trust Regions Improve Reinforcement Learning for Large Language Models](#)

Problem statement

The well-known **proximal policy optimization** algorithm can be expressed via simple and explicit formulas for actor and critic losses as follows:

$$\begin{aligned}
 J_\pi(s, a) &= \frac{\pi_\theta(a|s)}{\pi_\theta^{\text{old}}(a|s)} \cdot A^{\text{GAE}(\gamma, \lambda)}(s, a), \\
 J_\pi^{\text{clipped}}(s, a) &= \text{clip} \left(\frac{\pi_\theta(a|s)}{\pi_\theta^{\text{old}}(a|s)}, 1 - \text{cliprange}, 1 + \text{cliprange} \right) \cdot A^{\text{GAE}(\gamma, \lambda)}(s), \\
 L_{\text{policy}} &= -\frac{1}{B} \sum_{\text{batch}} [\min(J_\pi(s, a), J_\pi^{\text{clipped}}(s, a)) + \beta \cdot H(s)], \\
 l(s) &= [V_\phi(s) - \hat{V}(s)]^2, \\
 l^{\text{clipped}}(s) &= [V_\phi^{\text{old}}(s) + \text{clip}(V_\phi(s) - V_\phi^{\text{old}}(s), -\text{cliprange}, \text{cliprange}) - \hat{V}(s)]^2, \\
 L_{\text{critic}} &= \frac{1}{B} \sum_{\text{batch}} \max(l(s), l^{\text{clipped}}(s)), \\
 L_{\text{total}} &= L_{\text{policy}} + \gamma \cdot L_{\text{critic}}.
 \end{aligned} \tag{1}$$

Here we used default notations for actor $\pi_\theta(a|s)$, critic $V_\phi(s)$, critic targets $\hat{V}(s)$, entropy $H(s)$, and **generalized advantage estimation** $A^{\text{GAE}(\gamma, \lambda)}(s)$. GAE and critic targets can be calculated over sampled rollouts by the following way:

$$\begin{aligned}
 A^{\text{GAE}(\gamma, \lambda)}(s_t) &= \sum_{l=0}^{T-1-t} (\gamma\lambda)^l \delta_{t+l}^V, \quad \delta_{t+l}^V = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l}), \quad t \in [0, T), \\
 \hat{V}(s) &= A^{\text{GAE}(\gamma, \lambda)}(s) + V(s).
 \end{aligned} \tag{2}$$

Despite its universality, convergence velocity, efficiency in many issues, and wide spreading, PPO has one big problem: it requires a clipping procedure, which could have a strongly negative influence on stability. Moreover, PPO exploits min/max operations, which also can lead to sharp gradient changes.

Proposed solution

Clipping in the PPO paradigm is essential because the new policy must be close to the old policy at each optimization step. However, one can suggest a more convenient and mathematically correct way to satisfy the "closeness" condition. This logic may be based on keeping **KL-divergence** between the new and old policy distributions bounded at each iteration:

$$\text{KL}(\pi_\theta(\cdot|s) \parallel \pi_\theta^{\text{old}}(\cdot|s)) \leq \epsilon. \tag{3}$$

Direct numerical expansion and inversion of the KL-divergence bounding limitation at every step is very computationally expensive, so one needs to develop some tricks. TROLL authors came up with a prosperous idea how to transform conditioned optimization (3) into a simple convex scalar maximization problem for each minibatch.

Thus, the TROLL method modifies the PPO actor loss in the following way:

$$\begin{aligned}
 \tilde{\pi}_\theta(\cdot|s) &= \text{softmax} \left\{ \begin{array}{l} \ln \pi_\theta(\cdot|s), \text{ if } \text{KL}(\pi_\theta(\cdot|s) \parallel \pi_\theta^{\text{old}}(\cdot|s)) \leq \epsilon, \\ \frac{\eta^* \cdot \ln \pi_\theta^{\text{old}}(\cdot|s) + \ln \pi_\theta(\cdot|s)}{\eta^* + 1}, \text{ otherwise.} \end{array} \right\}, \\
 \eta^* &= \underset{\eta}{\text{argmax}} \left\{ -\eta \cdot \epsilon - (\eta + 1) \cdot \ln \sum_a \exp \left[\frac{\eta \cdot \ln \pi_\theta^{\text{old}}(a|s) + \ln \pi_\theta(a|s)}{\eta + 1} \right] \right\}, \quad \text{s.t. } \eta \geq 0, \\
 J_\pi(s, a) &= \frac{\tilde{\pi}_\theta(a|s)}{\pi_\theta^{\text{old}}(a|s)} \cdot A^{\text{GAE}(\gamma, \lambda)}(s) - \alpha \cdot \text{KL}(\pi_\theta(\cdot|s) \parallel \text{stopgrad}[\tilde{\pi}_\theta(\cdot|s)] + \beta \cdot H(s)), \quad \alpha \approx 1, \\
 L_{\text{critic}} &= -\frac{1}{B} \sum_{\text{batch}} J_\pi(s, a).
 \end{aligned} \tag{4}$$

Environment

We have compared PPO and TROLL methods on two Atari games from [gymnasium](#): "Space Invaders" and "Breakout" (1). Both environments were wrapped to output normalized grayscale and frame-stacked images with fixed shape (4, 84, 84). These games have discrete action spaces with $n = 6$ and $n = 4$ respectively.



Space Invaders



Breakout

Fig. 1: Atari games

Model

We have chosen a small custom CNN neuron net with two heads (2). The first head predicts logits for the actions' distribution, and the second one estimates values.

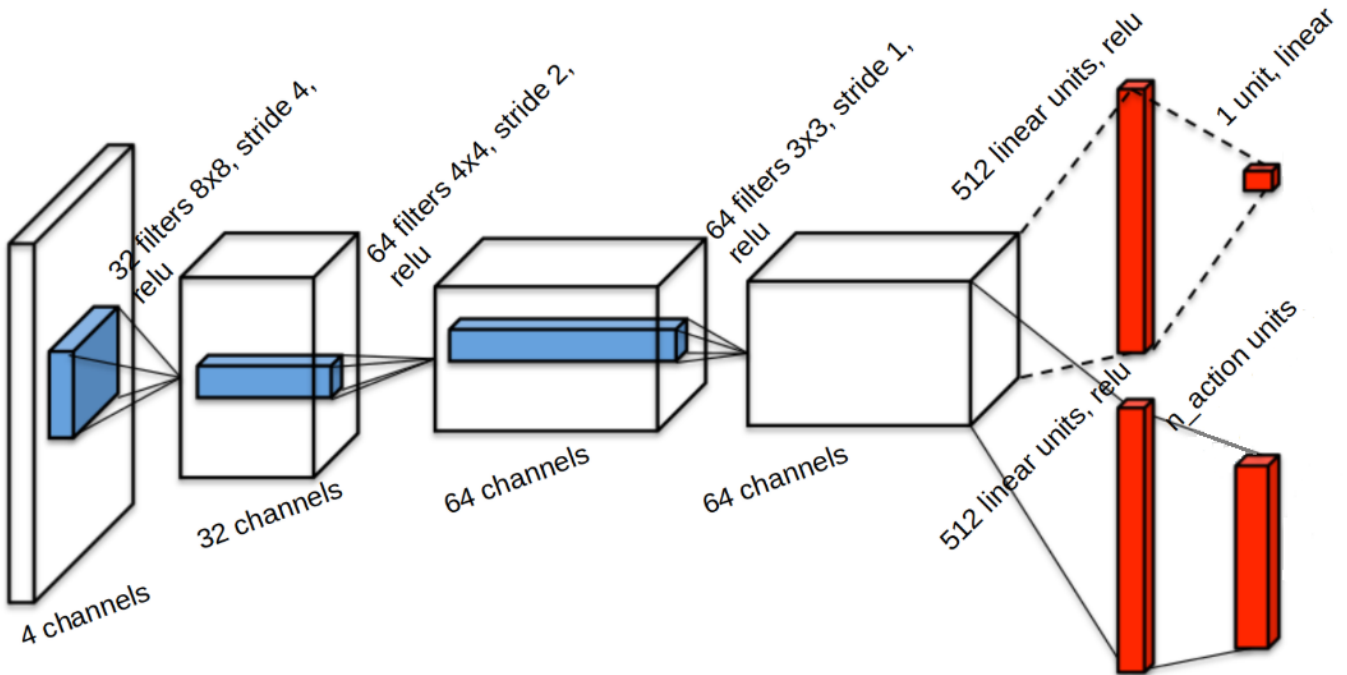


Fig. 2: CNN model

Experiment and Results

We've fitted our model to play "Space Invaders" and "Breakout" via PPO and TROLL algorithms independently several times to obtain relevant statistics for train curves. These results are presented on the plot (3).

Firstly, it's important to mention that both algorithms successfully learned non-dummy game strategies in all experiments.

Secondly, many pre-experimental attempts have demonstrated that the "Breakout" environment requires much more exploration to converge. This fact leads to more aggressive entropy regularization for the "Breakout" pipeline.

Thirdly, the given graphics show that TROLL is approximately equivalent to PPO on "Space Invaders" and a little bit faster on "Breakout". Moreover, TROLL reward and loss curves are less noisy and more stable in both games.

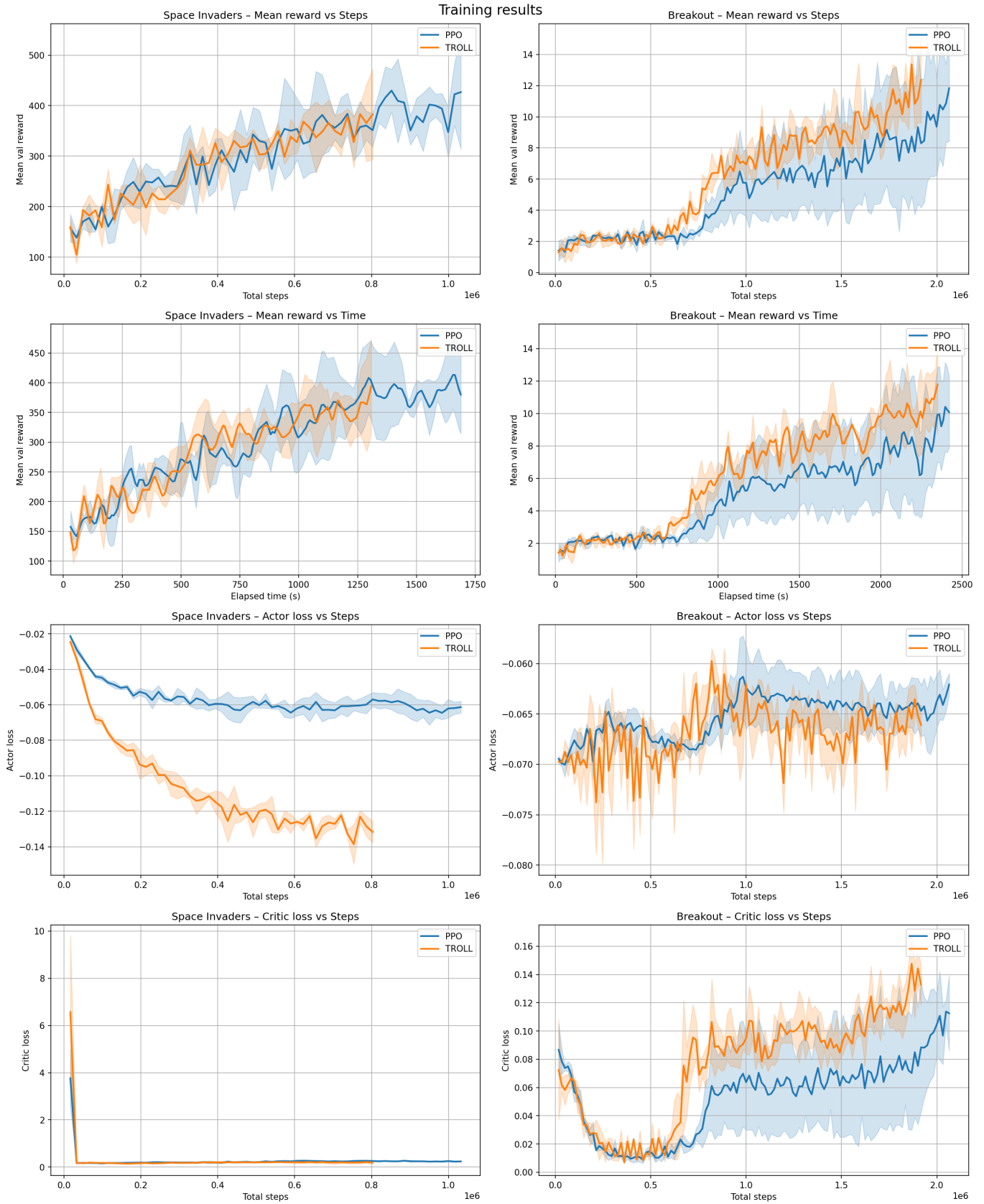


Fig. 3: Training results

Discussion

Based on the provided results, one can conclude that the TROLL algorithm is a bit faster, more stable, and more sample efficient than ordinary PPO in complex environments with a poor reward signal. The simple games training process is not strongly sensitive to a choice between two mentioned algorithms.